

Inteligência Artificial | FGA0021 | Turma 01 | 2026.1

Professor: Dr. Fabiano Araujo Soares

Aluno: Bruno Cruz Garcia Rosa

Matrícula: 222006605

Busca cega e busca informada

1. Introdução

Problemas de busca consistem em encontrar um caminho válido entre um estado inicial e um objetivo, respeitando determinadas regras e restrições.

Neste projeto foi desenvolvido um sistema de geração automática de labirintos, no qual dois algoritmos de busca foram aplicados para encontrar caminhos entre um ponto inicial e um ponto final:

- Breadth-First Search (Busca em Largura) (**BFS**)
- A* (**A-Star**)

O objetivo principal consiste em comparar o comportamento de um algoritmo de busca cega e outro de busca informada em diferentes estruturas de labirinto, analisando métricas como:

- tempo de execução;
- quantidade de nós visitados;
- tamanho do caminho encontrado.

Além disso, foi utilizada a biblioteca **Pygame** para a construção da interface gráfica e visualização da exploração dos algoritmos no labirinto, permitindo acompanhar visualmente a expansão dos nós e o caminho encontrado.

Para execução correta do projeto, é necessária a instalação prévia da biblioteca Pygame através do comando:

- `pip install pygame`

ou:

- `pip install pygame-ce`

2. Definição do Problema

O problema abordado neste projeto consiste na resolução de labirintos gerados proceduralmente através da aplicação de algoritmos de busca. O ambiente é representado como uma matriz bidimensional composta por células livres e paredes, contendo um ponto inicial e um ponto objetivo.

O agente deve encontrar um caminho válido entre o estado inicial e o estado objetivo, respeitando as restrições de movimentação impostas pelo labirinto.

2.1. Formalização do Problema:

- **Estado Inicial:** O estado inicial corresponde à posição marcada como “S” (Start) no labirinto, representando o ponto de partida do agente dentro da matriz.
- **Ações:** As ações disponíveis consistem na movimentação do agente nas quatro direções cardeais (cima, baixo, direita, esquerda). Uma movimentação só é permitida caso a célula de destino não seja uma parede e esteja dentro dos limites do labirinto.
- **Objetivo:** O objetivo consiste em alcançar a posição marcada como “E” (End), representando a saída do labirinto.
- **Custo do caminho:** Cada movimentação possui custo unitário igual a 1. Dessa forma, o custo total do caminho corresponde à soma das movimentações realizadas desde o estado inicial até o objetivo.

2.2. Complexidade do Ambiente

Os ambientes utilizados nos experimentos foram desenvolvidos de forma procedural, permitindo a criação de labirintos com diferentes níveis de complexidade e estruturas distintas. Foram desenvolvidos labirintos com múltiplos caminhos e ciclos, utilizando a adição de loops e conexões extras entre regiões do mapa. Essa modificação aumentou significativamente a complexidade do ambiente, uma vez que os algoritmos passaram a lidar com um maior número de possibilidades de movimentação e caminhos alternativos.

Além disso, foram realizados testes com diferentes dimensões de labirinto, incluindo ambientes quadrados e retangulares, permitindo analisar como o crescimento do espaço de estados impacta diretamente o desempenho dos algoritmos de busca.

Observou-se que a estrutura do ambiente influencia fortemente a eficiência dos algoritmos. Em labirintos muito fechados, contendo poucos caminhos alternativos, o algoritmo A* apresentou desempenho semelhante ao BFS, pois a heurística utilizada possui pouca capacidade de direcionamento. Já em ambientes mais abertos e com maior quantidade de caminhos possíveis, o A* conseguiu explorar menos estados e apresentar melhores resultados.

3. Algoritmos Desenvolvidos

3.1. Geração do Labirinto

Para os experimentos, foi desenvolvido um sistema de geração procedural de labirintos utilizando matrizes bidimensionais compostas por paredes e caminhos livres.

Inicialmente, a função `create_maze(rows, cols)` cria uma matriz bidimensional utilizando os valores de altura (*rows*) e largura (*cols*) fornecidos como parâmetro. Nesse primeiro momento, toda a matriz é preenchida com paredes, representadas pelo caractere '#’.

Após a criação da estrutura inicial, o algoritmo utiliza uma estratégia baseada em **Depth-First Search (DFS)** para “escavar” caminhos dentro do labirinto. O processo começa em uma célula inicial e percorre recursivamente posições vizinhas ainda não visitadas, removendo paredes entre as células para criar caminhos conectados. A cada etapa, os vizinhos são embaralhados aleatoriamente, fazendo com que cada execução gere um labirinto diferente. Como consequência, o algoritmo produz estruturas proceduralmente únicas, mesmo utilizando as mesmas dimensões. A utilização da DFS na geração do labirinto garante que todos os caminhos criados permaneçam conectados, assegurando que exista pelo menos um caminho válido entre o ponto inicial e o objetivo final.

Ao término da geração, o algoritmo marca:

- “S” (Start), representando o ponto inicial;
- “E” (End), representando a saída do labirinto.

Para aumentar a complexidade do ambiente, foi desenvolvida a função *add_loops(maze, loop_density)*, responsável por adicionar caminhos alternativos ao labirinto. Essa função percorre posições específicas da matriz verificando paredes que podem ser removidas sem comprometer a estrutura do ambiente. Quando uma parede é removida, dois caminhos anteriormente separados passam a ficar conectados, criando ciclos e múltiplas rotas possíveis até o objetivo.

O parâmetro *loop_density* controla a quantidade de loops adicionados ao ambiente. Esse valor funciona como uma probabilidade percentual de determinadas paredes serem convertidas em caminhos livres.

Por exemplo:

- valores baixos geram labirintos mais fechados e lineares;
- valores altos produzem ambientes mais abertos e com maior

quantidade de caminhos alternativos. Essa etapa foi fundamental para os experimentos realizados, pois permitiu analisar como diferentes estruturas de ambiente influenciam diretamente o comportamento dos algoritmos de busca. Com os testes, foi descoberto que uma taxa de 3 - 5 % é ideal para um labirinto bem estruturado.

Por fim, a função *create_maze_with_loops(rows, cols, loop_density)* combina as etapas anteriores, gerando labirintos mais complexos e com diferentes densidades de caminhos alternativos.

3.2. Ferramentas para uso nas busca

Além dos algoritmos principais, foram desenvolvidas funções auxiliares responsáveis por simplificar operações recorrentes durante a execução das buscas.

A função *find_symbol(maze, symbol)* percorre toda a matriz do labirinto em busca de símbolos específicos, como:

- “S” para o ponto inicial;

- “E” para o objetivo final.

Essa função permite localizar dinamicamente os estados importantes do problema sem depender de posições fixas.

A função *is_valid_move(maze, row, col)* verifica se uma movimentação é válida. Para isso, o algoritmo garante que:

- a posição esteja dentro dos limites da matriz;
- a célula não seja uma parede.

Essa verificação impede que os algoritmos tentem acessar posições inválidas ou atravessem obstáculos do labirinto.

A função *get_neighbors(maze, position)* retorna todos os estados vizinhos acessíveis a partir de uma determinada posição. O algoritmo verifica as quatro direções cardeais (cima, baixo, esquerda e direita), utilizando *is_valid_move()* para determinar quais movimentos são permitidos.

Essas funções foram utilizadas tanto pelo **BFS** quanto pelo **A***, padronizando o processo de exploração do ambiente.

3.3. Busca Cega - BFS

O algoritmo **BFS (Breadth-First Search)** foi utilizado como representante dos algoritmos de busca cega. Sua implementação foi realizada através da função *bfs(maze)*, utilizando uma fila **FIFO** (First In, First Out) para controlar a ordem de exploração dos estados.

O algoritmo inicia sua execução a partir do estado inicial “S”, inserindo essa posição na fila. Em seguida, o BFS remove elementos da fila um a um, explorando todos os vizinhos válidos antes de avançar para níveis mais profundos do labirinto.

Esse comportamento faz com que o BFS explore o ambiente em “camadas”, visitando primeiro todos os estados mais próximos da origem, depois os mais distantes.

Durante a execução, o algoritmo:

- armazena os estados visitados;
- registra o pai de cada nó explorado;
- reconstrói o caminho final ao alcançar o objetivo.

Por utilizar custo uniforme para todas as ações, o BFS garante encontrar o menor caminho possível entre origem e destino.

Entretanto, essa abordagem também apresenta limitações.

Como o algoritmo não possui nenhuma informação heurística sobre a direção do objetivo, ele tende a explorar uma grande quantidade de estados, principalmente em ambientes extensos ou muito abertos.

Além disso, foi utilizada a função *mark_path(maze, path)*, responsável por marcar visualmente o caminho encontrado no labirinto após a conclusão da busca.

3.4. Busca Informada - A*

O algoritmo A* foi utilizado como representante dos algoritmos de busca informada. Sua implementação foi realizada através da função *astar(maze)*, utilizando uma fila de prioridade para selecionar os estados mais promissores durante a exploração.

O algoritmo utiliza a função heurística *heuristic(position, goal)*, baseada na **distância Manhattan**, calculada pela soma das diferenças absolutas entre linhas e colunas do estado atual e do objetivo. A função heurística permite direcionar a busca em direção ao objetivo, reduzindo potencialmente a quantidade de estados explorados em comparação ao BFS.

Durante os experimentos, observou-se que o desempenho do A* depende fortemente da estrutura do ambiente. Em labirintos muito fechados e com poucas alternativas de caminho, a heurística possui baixa capacidade de direcionamento, fazendo com que o algoritmo apresente desempenho semelhante ao BFS.

3.5. Pygame

Para facilitar a análise visual do comportamento dos algoritmos, foi utilizada a biblioteca Pygame.

A função *draw_maze(screen, maze)* é responsável por desenhar o labirinto na janela gráfica, representando paredes, caminhos livres, ponto inicial, objetivo e caminho encontrado. Ela utiliza cada caractere dentro da matriz para desenhar uma cor diferente da tela, criando o visual do labirinto.

A função *visualize_maze(maze, exploration_order, path)* realiza a animação da exploração dos algoritmos, permitindo visualizar a expansão dos nós visitados e o caminho final encontrado. Utilizando a ordem de exploração e resultado retornada pelos algoritmos de busca, ele pinta pixel a pixel, resultando na animação.

A utilização do Pygame tornou possível acompanhar visualmente as diferenças entre BFS e A*, auxiliando na compreensão do funcionamento dos algoritmos e na interpretação dos resultados obtidos.

3.6. Execução Principal (Main)

O arquivo principal do projeto é responsável pela geração do ambiente, execução dos algoritmos de busca e coleta das métricas utilizadas nos experimentos.

Inicialmente, é criado um labirinto procedural utilizando a função *create_maze_with_loops()*. Em seguida, são realizadas cópias independentes do ambiente para execução separada do BFS e do A*, garantindo que ambos os algoritmos atuem sobre o mesmo cenário.

Durante a execução, são coletadas métricas como:

- tempo de execução;
- quantidade de nós visitados;
- comprimento do caminho encontrado.

Por fim, os resultados são exibidos no terminal e também visualizados graficamente através do Pygame. Para uma melhor visualização, é recomendado o uso da função de animação de labirintos de no máximo 30x30, pois a animação se torna muito lenta à medida que a área do labirinto cresce.

4. Conclusão

Para avaliação dos algoritmos desenvolvidos, foram realizados experimentos utilizando diferentes tamanhos de labirinto e diferentes densidades de loops.

Os testes foram executados em ambientes 50x50 e 500x500. Além disso, foram utilizadas diferentes probabilidades de quebra de paredes, sendo essas: 3%, 10% e 30%.

Esses valores controlam diretamente a quantidade de caminhos alternativos presentes no labirinto, permitindo analisar como a estrutura do ambiente influencia o desempenho dos algoritmos BFS e A*.

Tabela 1 - Resultados obtidos em labirintos 50x50

Labirinto	Densidade (%)	Algoritmo	Tempo (ms)	Nós Visitados	Caminho
50x50	3	BFS	5,702	4821	343
50x50	3	A*	5,472	3361	343
50x50	10	BFS	5,161	6419	169
50x50	10	A*	2,752	1637	169
50x50	30	BFS	7,387	6008	177
50x50	30	A*	2,412	1219	177

Tabela 2 - Resultados obtidos em labirintos 500x500

Labirinto	Densidade (%)	Algoritmo	Tempo (ms)	Nós Visitados	Caminho
500x500	3	BFS	714.836	482849	2635
500x500	3	A*	827.106	337670	2635
500x500	10	BFS	694.947	491859	1719
500x500	10	A*	381.201	159534	1719
500x500	30	BFS	898.406	603006	1583
500x500	30	A*	525.857	179649	1583

Os resultados demonstram que a eficiência do **A*** depende fortemente da qualidade da heurística e da estrutura do ambiente. Em cenários com maior liberdade de movimentação, a busca informada consegue aproveitar melhor a estimativa heurística, tornando-se significativamente mais eficiente que a busca cega.

Figura 1 - Labirinto 30x30 - 3%, resolvido com **A***

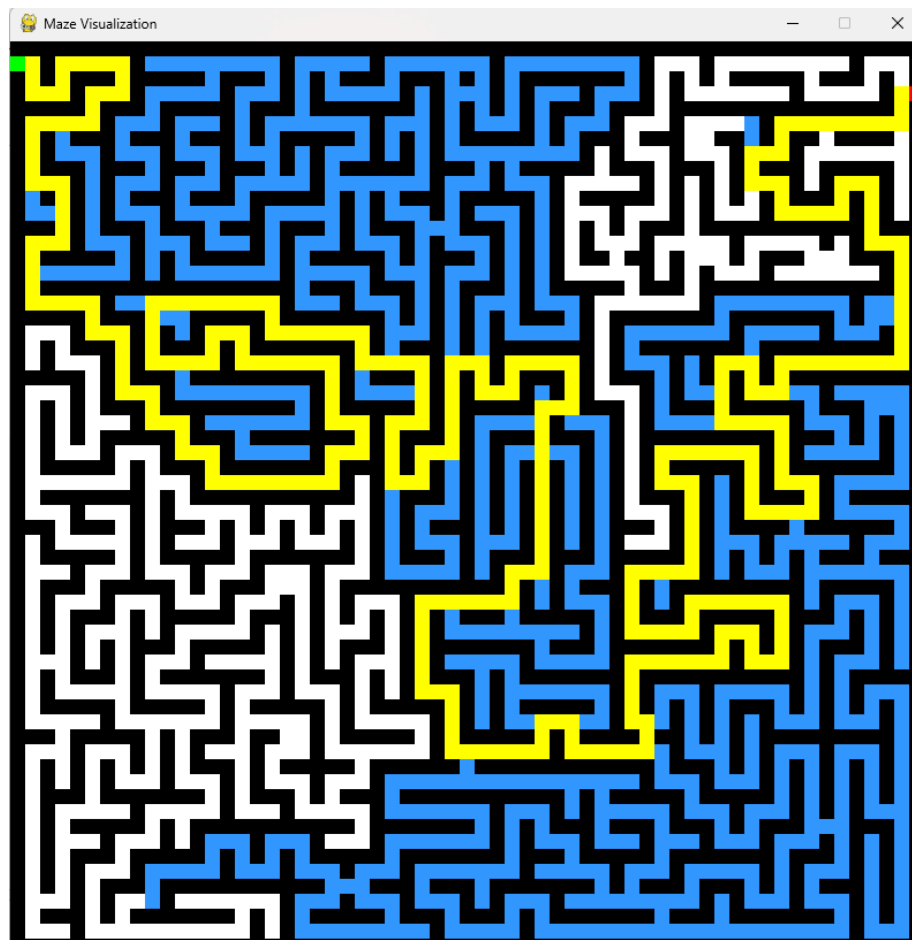
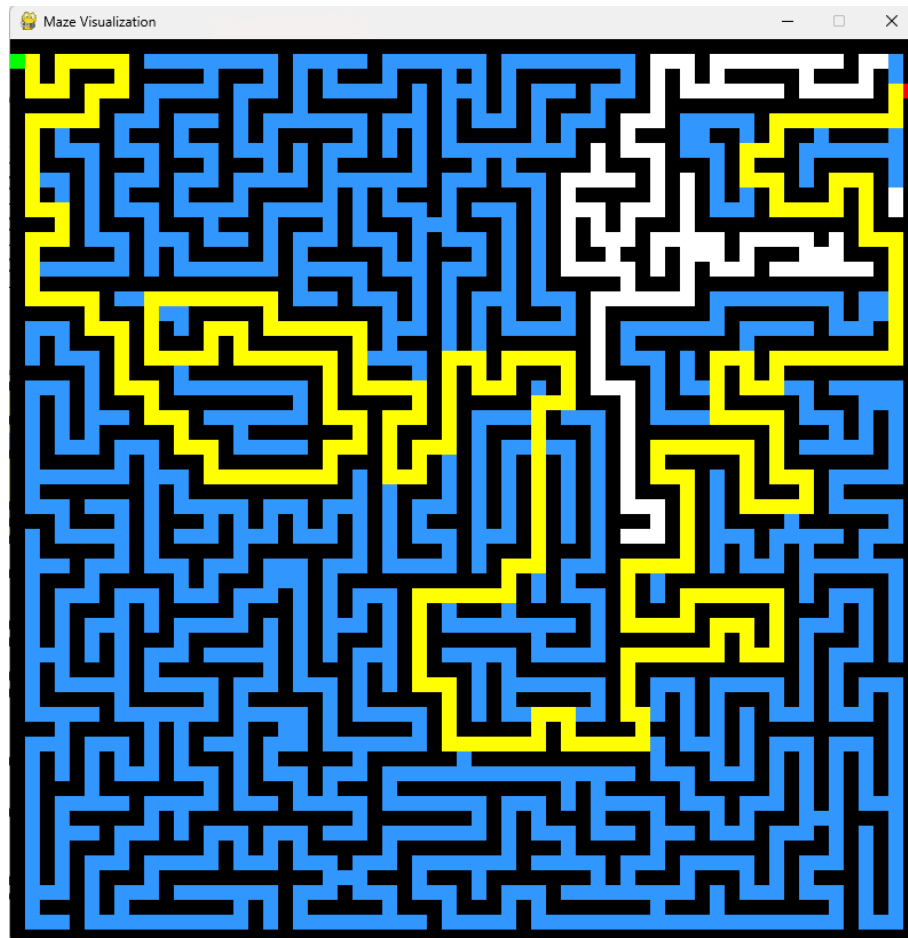


Figura 2 - Mesmo Labirinto, resolvido com **BFS**

De forma geral, ambos os algoritmos conseguiram encontrar o menor caminho possível em todos os experimentos realizados, apresentando o mesmo comprimento de solução para cada ambiente.

Entretanto, os resultados mostraram diferenças significativas na eficiência da exploração. Enquanto o **BFS** realiza uma expansão mais uniforme do espaço de estados, o **A*** consegue direcionar melhor sua busca em ambientes mais abertos, reduzindo a quantidade de estados explorados e melhorando o desempenho geral do algoritmo.